

ESILV x ALICE ML Workshop

General Project Guidelines

One-Month Applied AI Project

Faculty of Artificial Intelligence, Universiti Teknologi Malaysia

Level: Beginner to Intermediate AI Students

Workshop Context

This workshop is designed as a practical introduction to applied AI engineering. Students will work in teams on a real-world inspired problem and build a complete proof of concept: from data exploration and model development to explainability, backend integration, dashboard design and final presentation.

The emphasis is on building a system that is understandable, reproducible and useful for decision support. Students are expected to go beyond model training and demonstrate how their AI pipeline can be used, interpreted and communicated in a realistic application context.

Available Tracks

Students will work on one of the following project tracks.

Track A – AI-Powered Energy Demand Forecasting

Project title: Explainable Electricity Demand Forecasting Platform

Main objective: forecast the next 24 hours of electricity demand using historical energy, weather and market data.

Decision-support perspective: help energy planners understand future demand, model reliability and possible energy planning implications.

Optional advanced track: simplified energy mix optimization using forecasted demand.

Track B – AI-Powered Predictive Aircraft Maintenance

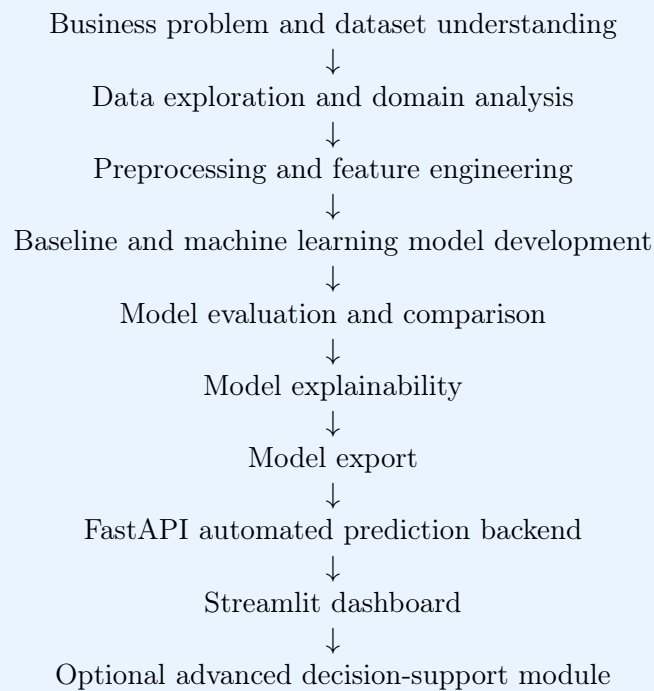
Project title: Explainable Engine Health and RUL Prediction Platform

Main objective: predict the Remaining Useful Life (RUL) of aircraft engines using sensor data.

Decision-support perspective: help maintenance teams identify engines that should be monitored, inspected or prioritized before failure.

Optional advanced track: risk-based maintenance prioritization.

Common Applied AI Framework



Team Format

Students will work in teams of **1 to 5 members**.

Each team is responsible for organizing its own internal workflow. The final submission must include a clear breakdown of each member's contribution.

Suggested areas of responsibility include:

- data exploration and preprocessing;
- model development and evaluation;
- explainability and interpretation;
- FastAPI backend development;
- Streamlit dashboard.

These roles are suggestions only. Teams may organize differently as long as all members contribute meaningfully.

Common Technical Architecture

Every team must deliver three connected technical components:

1. a notebook-based modeling workflow;
2. a FastAPI backend;
3. a Streamlit dashboard.

Notebook-Based Modeling Workflow

The notebook is the research, experimentation and training environment.

It should include:

- problem definition;
- data exploration;
- preprocessing decisions;
- feature engineering;
- baseline model;
- machine learning model training;
- model evaluation;
- explainability;
- model export.

The notebook should not only contain code. It should also include observations, explanations and technical decisions.

FastAPI Backend

The backend is the automated prediction service.

It must expose the finalized prediction pipeline through an API and apply the same preprocessing and feature-generation logic defined in the notebooks.

The backend must:

- load the exported model and preprocessing artifacts;
- validate input data structure;
- apply deterministic preprocessing steps required for prediction;
- run the trained model;
- return predictions and useful metadata to the frontend;
- handle invalid inputs with clear error messages.

The backend should automate the finalized pipeline designed and justified in the notebooks.

Streamlit Dashboard

The Streamlit dashboard is the user-facing product.

It must communicate with the FastAPI backend rather than directly running all prediction logic inside the dashboard.

The dashboard should guide the user from data understanding to prediction, explanation and decision-support interpretation.

The frontend must be modular. A single long Python file is not acceptable.

Model Export

The trained model must be exported so that it can be reused outside the notebook environment.

Students may use:

- `joblib`;
- `pickle`;
- another justified model serialization method.

The exported artifacts should include:

- the trained machine learning model;
- required preprocessing objects, such as scalers, encoders or feature-generation objects;
- selected feature names;
- useful model metadata.

The FastAPI backend must load these artifacts rather than retraining the model.

Common Deliverables

Each team must submit:

1. a GitHub repository;
2. notebooks covering data preparation, training, evaluation, explainability and export;
3. exported model and metadata;
4. FastAPI backend;
5. modular Streamlit dashboard connected to the backend;
6. README with setup and run instructions;
7. per-member contribution breakdown;
8. optional advanced decision-support module, if implemented.

Recommended Repository Structure

Teams may adapt the structure to their track, but the repository should remain clean and easy to run.

```
project-name/  
|-- notebooks/  
|   |-- 01_data_exploration.ipynb  
|   |-- 02_model_training.ipynb  
|   |-- 03_explainability_and_export.ipynb  
|-- backend/  
|   |-- main.py  
|   |-- schemas.py
```

```
|  |-- services/
|      |-- preprocessing.py
|      |-- prediction.py
|      |-- explainability.py
|      |-- optional_module.py
|-- frontend/
|   |--app.py
|   |--pages/
|   |-- components/
|   |-- utils/
|       |-- api_client.py
|-- models/
|   |-- model.pkl
|   |-- model_metadata.json
|-- data/
|   |-- processed/
|   |-- sample_test/
|-- requirements.txt
|-- README.md
|-- .gitignore
```

Raw data should not be committed. Teams should provide dataset download instructions instead.

Reproducibility Requirements

Teams must ensure:

- fixed random seeds for model training and stochastic steps when relevant;
- a pinned `requirements.txt`;
- a clear README with setup, training and run instructions;
- instructions to regenerate or load the exported model;
- a clean repository structure;
- no raw dataset committed to the repository;
- a per-member contribution breakdown.

A project that cannot be installed, run or understood from the README will be considered incomplete.

Advanced Optional Track

Teams that successfully complete the core system are highly encouraged to tackle the optional advanced track for an extra challenge.

This module will only be reviewed if the core system is already functional.

The optional module cannot compensate for:

- a weak or non-functional machine learning pipeline;
- a missing backend;

- a missing or disconnected dashboard;
- absent explainability;
- poor reproducibility.

The optional advanced track should add a decision-support perspective to the project and should be clearly connected to the model predictions.

Project Timeline

Week	Expected Progress
Week 1	Understand the business problem, load the dataset, explore the data, identify pitfalls, define the target and prepare the repository.
Week 2	Clean data, engineer features, implement a baseline, train first machine learning models and evaluate initial results.
Week 3	Improve models, add explainability, export the model, build the FastAPI backend and connect the prediction pipeline.
Week 4	Build the Streamlit dashboard, integrate explanations and decision-support outputs, optionally add the advanced module, finalize README.

Common Technical Stack

Core Stack

- Python;
- pandas;
- NumPy;
- scikit-learn;
- joblib or pickle;
- FastAPI;
- Uvicorn;
- Pydantic;
- python-multipart;
- Streamlit;
- requests;
- Plotly or Matplotlib.

Optional Libraries

Depending on the track, teams may also use:

- XGBoost or LightGBM;
- SHAP;
- OR-Tools;
- PuLP;
- TensorFlow or PyTorch;
- skforecast or sktime for time-series forecasting.

Final Note

A strong project should show:

- clear problem understanding;
- responsible data handling;
- meaningful model evaluation;
- explainable predictions;
- clean engineering structure;
- a dashboard that supports interpretation and decision-making.